

category: networking-protocol

tags: [tcp, networkstream, writeasync, timeout, heartbeat, watchdog, slow-reader, dos]

severity: high

created: 2026-07-26

updated: 2026-07-26

project-origin: RemoteControlTool (IPGS.RemoteControl.ZcuAgent)

TCP server: heartbeat/watchdog đặt cùng loop với WriteAsync không timeout → timeout không bao giờ fire; `NetworkStream.WriteTimeout` KHÔNG áp dụng cho async write

Tình huống gặp phải

TCP server stream màn hình (ZcuAgent, 1 client/lượt): capture loop vừa `WriteAsync(frame)` vừa kiểm tra heartbeat (`now - lastPong > PingTimeoutMs`) trong cùng vòng lặp. Audit phát hiện client "slow-reader" (đã auth nhưng ngừng đọc — cố ý hoặc kẹt mạng) chiếm luôn slot session duy nhất, chặn mọi client hợp lệ (DoS).

Triệu chứng / Lỗi

- WriteAsync backpressure chặn vô hạn khi client không đọc (TCP window đầy)
- Kiểm tra heartbeat 15s nằm SAU WriteAsync trong cùng loop → không bao giờ chạy tới
- Session không bao giờ bị đóng dù client đã "chết" — server kẹt vĩnh viễn

Nguyên nhân gốc rễ (Root Cause)

1. **Watchdog và write cùng 1 loop tuần tự** — write chặn thì code kiểm tra timeout phía sau chết theo. Timeout chỉ có ý nghĩa khi nó chạy ở execution context ĐỘC LẬP với thao tác cần giám sát.
2. **`NetworkStream.WriteTimeout` chỉ áp dụng cho sync `Write`** — hoàn toàn bị bỏ qua với `WriteAsync` (documented nhưng rất dễ quên). Set WriteTimeout cho code async = vô tác dụng âm thầm.

Giải pháp

```
// 1. Watchdog tách task riêng — chỉ đọc _lastPongTicks (Interlocked cả 2 phía)
async Task WatchdogAsync(CancellationTokens ct) {
    while (!ct.IsCancellationRequested) {
        await Task.Delay(1000, ct);
        var last = Interlocked.Read(ref _lastPongTicks);
        if (Environment.TickCount64 - last > PingTimeoutMs) return; // chỉ return →
    }
    WhenAny xử lý đóng
```

```
}

// 2. Mọi WriteAsync bọc CTS CancelAfter – convert timeout thành IOException
using var writeCts = CancellationTokensSource.CreateLinkedTokenSource(sessionCt);
writeCts.CancelAfter(TimeSpan.FromSeconds(10));
try { await stream.WriteAsync(buffer, writeCts.Token); }
catch (OperationCanceledException) when (!sessionCt.IsCancellationRequested) {
    throw new IOException("Write timeout – client không đọc dữ liệu.");
}

// 3. Đóng sạch: Task.WhenAny(captureTask, receiveTask, watchdogTask)
// → sessionCts.CancelAsync() → Task.WhenAll (nuốt OCE) → dispose
```

1. Tách watchdog PONG thành task riêng, giao tiếp qua field **Interlocked** (không lock chung với write path).
2. Mọi async write có deadline qua **CancelAfter**; phân biệt write-timeout với session-cancel bằng **when (!sessionCt.IsCancellationRequested)**.
3. **WhenAny** + cancel + **WhenAll** đảm bảo cả 3 task (capture/receive/watchdog) đóng sạch, không leak.

Áp dụng lại (How to reuse)

- Khi thấy heartbeat/keepalive/timeout check nằm TRONG cùng loop với thao tác I/O blocking → tách ngay ra task riêng.
- Mọi **WriteAsync/ReadAsync** trên **NetworkStream** phía server công khai → luôn có **CancelAfter** (write đặc biệt quan trọng — read thường đã có heartbeat cover).
- Grep **WriteTimeout|ReadTimeout** trong code async → các set này vô tác dụng, thay bằng CTS.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Watchdog CHỈ nên return/cancel — không tự ghi socket (tránh 2 writer tranh chấp stream).
- ⚠ Cancel do timeout và cancel do session-shutdown đều ném **OperationCanceledException** — PHẢI phân biệt bằng **when (!sessionCt.IsCancellationRequested)** để log đúng bản chất.
- ⚠ **Environment.TickCount** (int) wrap sau ~24.9 ngày — dùng **TickCount64**.

Tham chiếu

- Project liên quan: **IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs** (fix L3, commit **1ab4f03**)
- GOTCHAS repo RemoteControlTool: G013